2026 IEEE 2nd International Conference on Quantum Photonics, Artificial Intelligence, and Networking (QPAIN)
16 – 18 April 2026, Chittagong, Bangladesh

Design and Implementation of a Low-Cost Two-Wheeled Self-Balancing Robot Using PID Controller

Md. Rakibul Hasan Nishat[1], S M Shafayet Jamil[2], Md. Meraj Ahmed[3*], Ezazul Haque[4]
[1,2,3*,4]Department of Mechatronics Engineering, Rajshahi University of Engineering & Technology
Emails: [1]rakibulnishat1864@gmail.com, [2]shaf1908006@gmail.com, [3*]meraj@mte.ruet.ac.bd, [4]ehsumit619@gmail.com

***Abstract*—Self-balancing robots (basically inverted pendulum systems) are an efficient system upon which to explore real-time feedback control and system dynamics. The paper advances the design, implementation, and experimental validation of a low-cost self-balancing robot made of two wheels and developed using an Arduino Nano microcontroller. The system employs an inertial measurement unit (IMU) of type MPU6050 to estimate the tilt, and the raw gyroscopes are filtered using a low-pass filter in order to reduce the high-frequency noise. To achieve the real-time balance of the robot, a discrete- time Proportional-Integral-Derivative (PID) controller is used that generates corrective motor command. The prototype has a rise time of 0.45 s and a settling time of 0.77 s after an initial tilt of 8°. The work highlights the practicality of PID control for educational and prototyping applications, while also acknowledging limitations such as sensor drift and fixed-gain operation. The low-cost and modular substructure provides a convenient base for advanced control methods and autonomous performance of the robotic balancing system.**

***Index Terms*—Self-Balancing Robot, PID Control, Inverted Pendulum, Arduino, MPU6050, Low-Cost Robotics, Real-Time Control, Low-Pass Filter, Embedded System, Educational Platform.**

I. Introduction

Self-balancing robots, commonly modeled as inverted pendulum systems, are becoming an essential platform for research in control engineering, robotics, and embedded systems. Their unstable nature makes them ideal for studying real-time feedback control, sensor fusion, and system dynamics.

The paper provides a design, implementation, and experimental validation of self-balancing robot constructed on a low-cost Arduino platform. It utilizes the gyro-accelerator sensor MPU6050 which measures the angular velocity. The filtered gyroscopes are used to estimate the tilt angle in real time using this robot. To maintain the motor outputs, a PID controller is then used to adjust the motor outputs in response to an error between the desired upright setpoint and the measured angle. The latest advances in balancing robots have assumed more than basic PID control implementation, and now elaborate balancing systems use Linear Quadratic Regulators (LQR), state-space controllers, machine learning programs, and advanced sensor fusion algorithms [1], [2], [3]. In contemporary systems it is usually interested in control parameter optimization with genetic algorithms or neural networks to be capable of achieving quality stability in a variety of conditions [4]. Our project aims first and foremost at closing the gap between theory and practice in control by designing and building a full-fledged self-balancing robot that will be affordable and of educational significance.

It is the work filled with a number of novelties, including affordable construction, the demonstration of the principles of PID control clearly, and modularity, which could receive improvements later. These features make it suitable to the educational laboratory, robotics, and personal studying projects. Besides, the effective realization provides a basis to see more advanced approaches to control in the sphere of robotics and mechatronics. The main contributions of this paper are,

1) A control architecture that achieves stable two-wheeled balancing using only gyroscopic feedback integrated for state estimation and a discrete-time PID controller.
2) The prototype achieves stable balancing performance at an ultra-low total cost of approximately $50-60.
3) It introduces an efficient filtering method for gyro noise reduction and angle estimation suitable for microcontrollers with limited computational resources.

II. Literature Review

Early research established the theoretical foundations of self-balancing systems by the classical control method, including PID, LQR, and state-space control, showing that real-time feedback is necessary to achieve the stability in the process of self-balancing systems and control [5], [6].

Several research studies have focused on PID-based control schemes because of their simplicity, strength, and the fact that they can be readily implemented on low-cost embedded systems. Savithri et al. [7] demonstrated that properly tuned PID controllers can achieve stable balancing performance for two-wheeled robots under moderate disturbances. In a comparable manner, Shekhawat et al. [8] introduced a self-balancing robot using an Arduino microcontroller and MPU6050 sensor, demonstrating that PID control combined with gyro-based angle estimation is sufficient for real-time stabilization.

In order to enhance performance and robustness and introduce functionality, major research has been done on advanced control strategies. Studies such as Lau et al. [9] indicated better results in the maximum initial tilt angle and the smoother control outputs with LQR. Similarly, Liao et al. [10] showed that LQR-based controllers provide improved stability margins compared to PID control. Furthermore, Saleem et al. [11] show intelligent control methods such as Fuzzy Logic Controllers (FLC) have been implemented to manage the system's nonlinearities [12]. Recent works have also explored the application of artificial intelligence and machine learning methods to

979-8-3315-4990-9/26/$31.00 ©2026 IEEE

balance control [3]. Reinforcement learning–based controllers have been applied to self-balancing robots to enable self-tuning and adaptive behavior [13].

Sensor accuracy and noise reduction are very critical in the self-balancing robots. Inertial Measurement Units (IMUs), in particular the MPU6050, are liable to frequent use because of their low price and inbuilt gyroscope and accelerator [14]. Fan et al. [15] and Ussayeva et al. [16] suggested sensor fusion methods to integrate gyroscope and acceleration data in an attempt to enhance orientation. However, raw IMU data is often noisy, which can degrade control performance. To overcome this, scholars have implemented filtering methods such as low-pass filters, complementary filters, and Kalman filters for reliable angle estimation [17].

III. System Design & Architecture

A. Hardware Selection

The physical system integrates standardized electronic modules on a custom 3D-printed chassis. The component selection and specifications are summarized in Fig. 3 and Table I.

TABLE I: Hardware Components and Specifications

Component	Functionality	Specifications
Arduino Nano	System control & processing	ATmega328P, 16 MHz, 32 KB Flash, 2 KB SRAM
IMU Sensor (MPU6050)	Tilt angle sensing	3-axis accelerometer, 3-axis gyroscope, I²C interface
Motor Driver (L298N)	Drives DC motors with PWM control	Dual H-bridge, outputs of up to 2A, voltages 5V-35V
Actuator (Nidec 24H Motors)	Provides motion and balance correction	12V DC, 25-300 RPM, Torque 50 In.Lb (5.6 Nm)
HC-05 Bluetooth	Remote command	UART communication, 2.4 GHz
Li-ion Battery	Main system power	12V and 1600mAh

B. Overall System Architecture

As illustrated in Fig. 1, the proposed system follows a closed-loop embedded control architecture centered on an Arduino Nano microcontroller. The system starts by initializing the IMU (MPU6050) and continuously receives control commands from a mobile app by the HC-05 module. The real-time data of accelerators and gyroscopes are used to determine the tilt angle of the robot. A PID controller calculates the corrective control signal, which is then converted into PWM commands to control the motors through the L298N motor driver. A fail-safe system is installed to ensure safe and reliable working of the system by stopping the motors when the tilt angle goes beyond the allowable limit. The entire system is powered by a 12 V Li-Po battery.

C. System Workflow

The workflow of the self-balancing robot follows a predictive sequence of initialization, continuous control, and safety monitoring, as illustrated in Fig. 2.

D. Control Algorithm

Algorithm 1 is based on the idea of studying the difference between the desired upright angle ($\theta_{\text{setpoint}} = 0°$) and the calculated current tilt angle (θ), producing corrective motor torque in proportion to the difference.

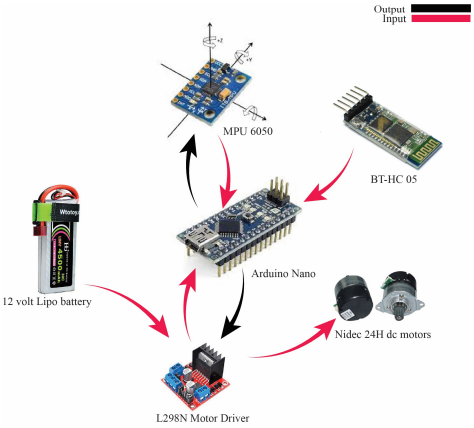


Fig. 1: Overall System Architecture

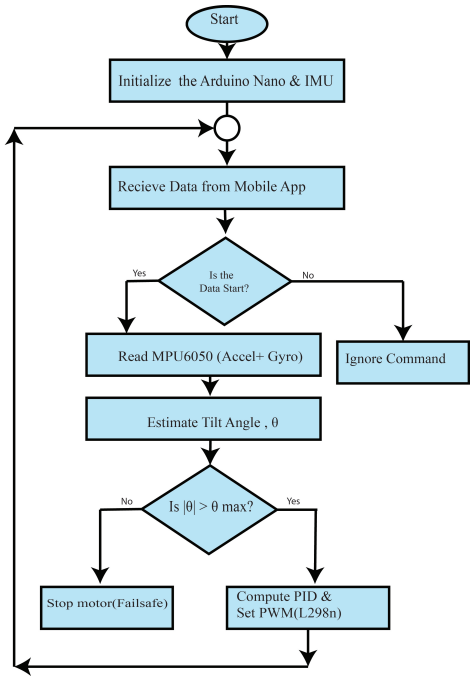


Fig. 2: Workflow Diagram

1) Initialization and Calibration Phase: The Arduino Nano operates the initialization sequence defined in the setup function. The gyroscope performs stationary bias calibration by averaging 200 samples.

$$gx_{\text{bias}} = \frac{1}{N}\sum_{i=1}^{N} gx_{\text{raw}}[i], \quad N = 200$$

where, $gx_{\text{raw}}[i]$ represents sequential gyroscope readings sampled every 5 ms.

2) Sensor Data Acquisition: The MPU6050 gyroscope provides raw 16-bit angular velocity.

$$gx_{\text{raw}} = \text{mpu.getRotation()}$$

3) Bias Correction: The raw gyroscope reading gx_{raw} is corrected by subtracting the pre-calibrated stationary bias.

$$gx_{\text{corrected}} = gx_{\text{raw}} - gx_{\text{bias}}$$

4) Low-Pass Filtering: A first-order infinite impulse response (IIR) filter attenuates high-frequency noise:

$$\text{filteredX(k)} = \alpha \cdot gx_{\text{corrected}}(k) + (1-\alpha) \cdot \text{filteredX}(k-1)$$

where, $\alpha = 0.6$ represents the filter coefficient, chosen to balance noise rejection with phase delay.

5) Differential Time Calculation: The elapsed time since the previous iteration is computed:

$$\Delta t = \frac{t_{\text{current}} - t_{\text{last}}}{1000} \quad \text{(seconds)}$$

where,
t_{current} and t_{previous} are consecutive time readings.

6) Angular Velocity Conversion: Filtered digital values are converted to physical units using the MPU6050 sensitivity constant.

$$\omega_x = \frac{\text{filteredX}}{131.0} \quad \text{(degrees per second)}$$

7) Tilt Angle Estimation: The tilt angle $\theta(k)$ is estimated through Euler integration:

$$\theta(k) = \theta(k-1) + \omega_x(k) \cdot \Delta t$$

where Δt is the time interval between consecutive iterations, typically 10 ms.

8) PID Control Process: The discrete-time PID controller calculates the corrective output.

$$e(k) = \theta_{\text{setpoint}} - \theta(k), \quad \theta_{\text{setpoint}} = 0^\circ$$

$$I[k] = I(k-1) + e(k) \cdot \Delta t$$

$$D(k) = \frac{e(k) - e(k-1)}{\Delta t}$$

$$u(k) = K_p \cdot e(k) + K_i \cdot I(k) + K_d \cdot D(k)$$

with empirically tuned parameters $K_p = 18.2$, $K_i = 0$, $K_d = 3.8$.

9) Motor Actuation: The control signal is mapped to PWM duty cycle and direction.

$$\text{PWM}[k] = \min\left(|u(k)|, 255\right)$$

$$\text{Direction} = \begin{cases} \text{Forward} & \text{if } u(k) > 0 \\ \text{Reverse} & \text{otherwise} \end{cases}$$

$$\text{Delay} = 10 \text{ ms} \quad \text{(100 Hz update rate)}$$

E. Cost Analysis

The price list, which can be viewed in Table II, reveals that all the parts could be obtained to create the whole system within about 5280 BDT, or 43.25$.

F. Comparative Analysis of Existing Systems

The self-balancing robot design presented above in Table III is found to represent a trade-off between cost, complexity, and capability. Our system is affordable and aims at minimalism and cheapness but it is balanced and offers functionality at around 1/9th the cost of the more complex systems.

Algorithm 1 Main Control Loop

procedure MAINCONTROLLOOP
  $g_x^{raw}, g_y^{raw}, g_z^{raw} \leftarrow$ mpu.getRotation() ▷ Read IMU data
  $g_x^{corr} \leftarrow g_x^{raw} - g_x^{bias}$ ▷ Bias removal
  $g_x^{filt}(k) \leftarrow \alpha g_x^{corr}(k) + (1-\alpha) g_x^{filt}(k-1)$ ▷ Low-pass filter
  $\Delta t \leftarrow (t_{current} - t_{previous})/1000$ ▷ Sampling time (s)
  $\omega_x(k) \leftarrow g_x^{filt}(k)/131.0$ ▷ Convert to °/s
  $\theta(k) \leftarrow \theta(k-1) + \omega_x(k)\Delta t$ ▷ Angle integration
  $e(k) \leftarrow \theta_{set} - \theta(k)$ ▷ Control error
  $I(k) \leftarrow I(k-1) + e(k)\Delta t$ ▷ Integral term
  $D(k) \leftarrow \dfrac{e(k) - e(k-1)}{\Delta t}$ ▷ Derivative term
  $u(k) \leftarrow K_p e(k) + K_i I(k) + K_d D(k)$ ▷ PID output
  if $u(k) > 0$ **then**
    set_motor_forward()
  else
    set_motor_reverse()
  end if
  $PWM(k) \leftarrow \min(|u(k)|, 255)$
  set_pwm($PWM(k)$)
  $t_{previous} \leftarrow t_{current}$
  delay(10) ▷ 100 Hz control loop
end procedure

TABLE II: Hardware Components and Specifications

Component	Quantity	Cost (BDT)	Cost (USD)
Arduino Nano	1	400	4
IMU Sensor (MPU6050)	1	160	1.5
Motor Driver (L298N)	1	120	1
Actuator (Nidec 24H Motors)	1	1200	10
HC-05 Bluetooth	1	150	1.35
Li-ion Battery	1	1450	11
3D printed parts	1	900	7.5
Other Accessories	1	700	5.8
Total		5280 BDT	43.25$

IV. IMPLEMENTATION AND PROTOTYPE

A. Hardware Integration

It is integrated with the hardware architecture and has an Arduino Nano as a central controller, which is connected to an MPU6050 IMU through the I²C protocol to measure tilt angles in real-time. The motor actuation is provided by a motor driver module (L298n) by receiving PWM and direction signals via the microcontroller to move the DC motors. One of the Bluetooth HC-05 modules is attached to the mobile application by serial communication, which allows it to control wirelessly. The entire system is powered by a 12 V LiPo battery. The hardware integration is shown in Fig. 3.

B. Software Implementation

The control algorithm was implemented in the Arduino IDE environment using C++, on a discrete-time PID control algorithm executing at 100 Hz. The MPU6050 library managed sensor communication, while the core loop sequentially performed gyro data reading, bias correction, low-pass filtering, and numerical integration to estimate the tilt angle.

TABLE III: Comparative Analysis of Self-Balancing Robot Systems

Aspect	Proposed System	PID-Based Systems [7], [8]	LQR/Advanced Control [1], [9], [10]	Intelligent Control [11]–[13]
Control Strategy	Discrete-time PID with gyro-only state estimation	Classical PID (often with Kalman filtering)	Linear Quadratic Regulator (LQR), state-space control	Fuzzy logic, adaptive PID, reinforcement learning
State Estimation	Gyroscope integration with low-pass filtering	Sensor fusion (gyroscope + accelerometer)	Kalman filter-based estimation	Advanced filtering and machine learning models
Processor	Arduino Nano	Arduino Mega / UNO	STM32, Raspberry Pi	Raspberry Pi
Primary Sensor	MPU6050	MPU6050	BMI160, ICM-20948	IMU, camera-based systems
Complexity	Low	Moderate	High	Very High
Cost	43.25 USD	80–150 USD	150–300 USD	$\geq$ 300 USD
Limitations	Gyro drift limits long-term stability; no velocity control	Limited by linear controller structure	Requires accurate system model; non-adaptive gains	Requires large datasets and training effort

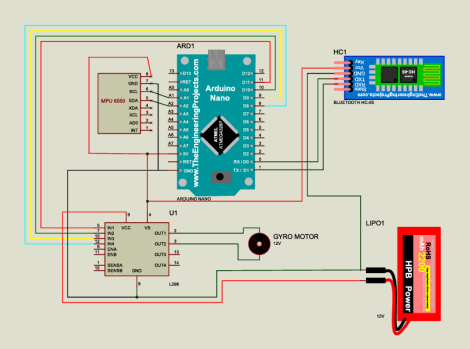


Fig. 3: Hardware Schematic Diagram

C. Mobile Application Interface

To monitor and control remotely as shown in Fig. 4, a specific mobile application was downloaded from the Google Play Store. Our emphasis has been on the development of the system, and hence we chose this ready-made app as opposed to developing it. The application creates a smooth wireless connection between the smartphone of the user and the HC-05 Bluetooth component of the robot, which forms the main interface to give commands and control telemetry.

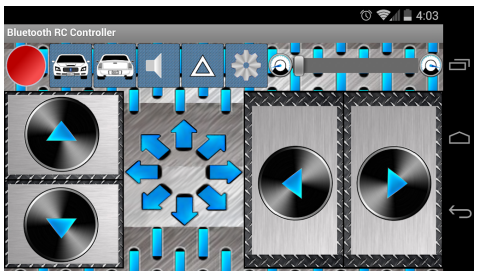


Fig. 4: Mobile Application Interface

D. Physical Assembly

The mechanical assembly of our robot took a methodical process in order to achieve appropriate alignment, structural integrity, and optimum weight distribution, as demonstrated in Fig. 5.

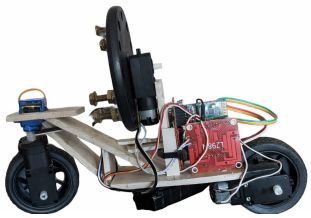


Fig. 5: Physical Prototype

V. Result Analysis & Performance Measurement

A. PID Tuning Methodology

An empirical iterative tuning method was used to obtain the gains of the PID. First, proportional gain was progressively raised up to the point when the system responded fast yet oscillated. The gain of the derivatives was then compensated to suppress oscillatory behavior. Lastly, to remove steady-state error but maintain stability, a small integral gain was added. The final tuned values were K_p = 18.2, K_i = 0.5, K_d = 3.8.

B. Performance Parameter of PID Controller

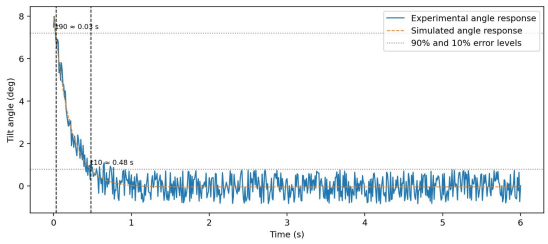
1) Rise-Time Response: The rise-time response plot 6a illustrates how the tilt angle converges from an initial 8° deviation towards the upright position under the tuned PID gains ((K_p = 18.2, K_i = 0.5, K_d = 3.8)). Rise time is defined here as the interval required for the error magnitude to decrease from 90% to 10% of its initial value. From the response, the rise time is approximately 0.45s.

2) Settling-Time Response: The settling-time plot 6b highlights the behavior of the tilt angle with respect to a ±2% error band around the desired upright position. Settling time is taken as the instant after which the response remains within this band and does not leave it again. For the considered controller and initial 8° disturbance, the settling time is approximately 0.77s.

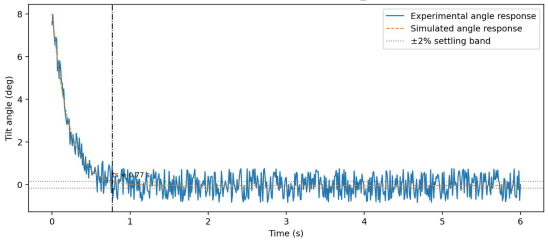
3) Overall Performance and Stability: The overall performance and stability 7 plot compares the experimental angle trajectory with the simulated response and the corresponding control effort. The angle signal converges smoothly towards zero without sustained oscillations or divergence, while the control effort remains bounded and decays over time.

C. Comparative Analysis of PID, PI, and PD Controllers

The robot was experimented with three different controller configurations, namely the full Proportional-Integral-



(a) PID Rise-Time Response



(b) PID Settling-Time Response

Fig. 6: PID response characteristics: (a) Rise time and (b) Settling time.

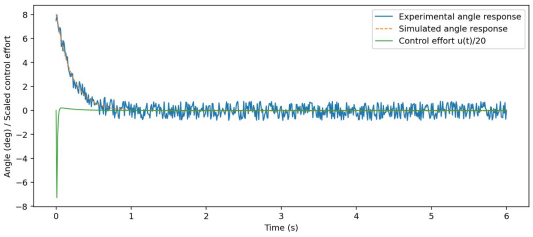


Fig. 7: Stability Curve

Derivative (PID), Proportional-Integral (PI) and Proportional-Derivative (PD) controllers to determine the effect of each control term on the performance of the system. The reaction of the individual configurations to a conventional step disturbance were captured and studied as demonstrated in Fig. 8 (PD), Fig .9, Fig. 10 (PID), and Table IV.

TABLE IV: Performance Comparison of Controllers

Controller	Rise Time (s)	Settling Time (s)	Overshoot (%)	Steady-State Error
PD	0.42	1.2	15%	Small residual
PI	0.65	1.8	30%+	0°
PID	0.45	0.77	<10%	0°

1) Proportional-Derivative (PD) Controller: With $K_p = 18.2, K_d = 3.8 (K_i = 0)$, the PD controller was tested and exhibited good transient performance. It gave rapid disturbance rejection and low overshoot (15%) with a fast settling time of about 1.2s as shown in 8. The derivative effectively smoothed out oscillations, and the result was stability. But a small, constant, steady-state error was present because of the lack of an integral term when subjected to a constant.

2) Proportional-Integral (PI) Controller: A PI controller was tested by setting $K_i > 0, K_d = 0$. This configuration successfully eliminated the steady-state error through integral action, driving the average tilt to precisely zero. However, as implied by the comparative trends in Fig.9, the absence of

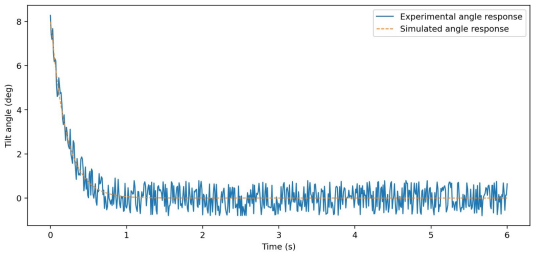


Fig. 8: Performance Analysis of PD Controller

derivative damping resulted in pronounced oscillatory behavior, a longer settling time, and significant overshoot

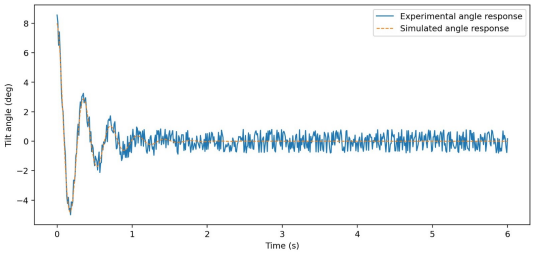


Fig. 9: Performance Analysis of PI Controller

3) Full Proportional-Integral-Derivative (PID) Controller: The entire PID controller was tested, and all three gains were on. Fig. 10 shows that it behaves like a PI and PD structure in its response. The integral term was used to remove the offset of steady state, whereas the derivative term supplied the required damping. This provided a zero steady-state error response together with damped oscillations.

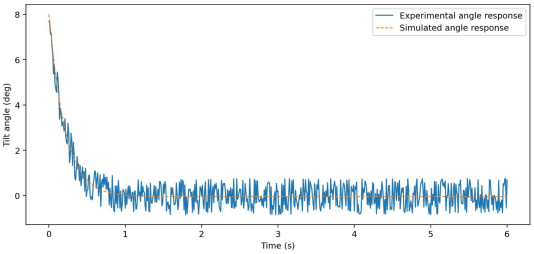


Fig. 10: Performance Analysis of PID Controller

D. Low-Pass Filter Performance

Fig. 11 showed the efficacy of the first-order low-pass filter. The raw value exhibits a significant high-frequency noise, whereas the filtered value at the value of ($\alpha = 0.6$) displays a much smoother curve. This processing has an approximation noise amplitude of 60%, which was significant in having a steady derivative signal and a reliable numerical integration of tilt angle determination.

VI. Discussion, Limitations & Future Work

The successful implementation of a two-wheeled, low-cost self-balancing robot has shown the feasibility of a discrete-time PID controller in combination with gyroscope-based state estimation. The system not only demonstrates stable balancing

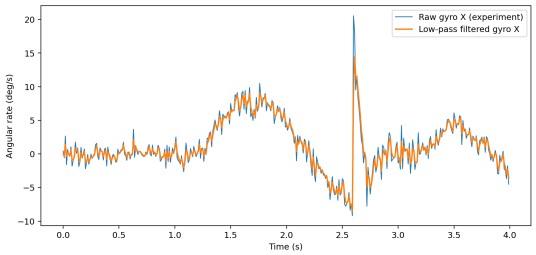


Fig. 11: Signal Conditioning Performance

performance but also provides a reproducible and low-cost laboratory framework for experiential learning in control engineering, robotics, and mechatronics. Students can visualize the trade-offs between overshoot, settling time, and steady-state error, reinforcing theoretical concepts from control systems courses. The system achieves stable balancing performance with a rise time of approximately 0.45 s and a settling time of 0.77 s after an initial disturbance of 8° tilt, which confirms a responsive and well-damped control loop. The overall cost of the prototype is approximately $43.25. The use of gyro-only angle estimation was adopted to minimize computational overhead and preserve real-time execution on the Arduino Nano platform. While complementary and Kalman filters offer better long-term stability. The comparative analysis of PD, PI, and PID controllers provides critical information on the role of each control term. The PD controller (K_p = 18.2, $K_d = 3.8$) showed excellent transient response with minimal overshoot (15%) but a steady-state error occurred due to the absence of integral action. On the other hand, the PI controller removed steady-state error without any derivative damping, which gave it a lot of oscillatory response and increased settling times. The complete PID controller was able to combine the virtues of both structures to produce zero steady-state error while maintaining well-damped and stable convergence, as illustrated in Fig. 10. The use of a first-order low-pass filter ($\alpha = 0.6$) to reduce gyroscope noise was found to be enough for reliable angle estimation without exceeding the computational power of the Arduino Nano microcontroller. Even though the main objectives were met, the proposed system is limited by a number of issues. The calculation of angles is based exclusively on the integration of gyroscopes, which causes cumulative drift that reduces long-term stability unless it is periodically recalibrated. The control law only controls the tilt error, and there is no velocity or position control, as no one controls the wheels, which limits autonomous navigation. The next steps in this platform will be to make it more robust, autonomous, and applicable with a number of improvements. Sensor fusion will be prioritized, integrating accelerometer information with the gyroscope through a complementary or Kalman filter to reduce drift and improve angle estimation accuracy.

VII. Conclusion

The paper has shown the whole design, implementation, and experimental validation of a two-wheel self-balancing robot at a low cost. The system has been able to provide stable upright balance with the help of a discrete-time PID controller and a state estimation pipeline based on a gyroscope on the Arduino Nano platform, at an overall price of about $43.25. The applied control architecture was able to attain a rise time of 0.45 s and a settling time of 0.77 s during an 8° tilt disturbance, confirming effective disturbance rejection and stable dynamic performance. Although the system has shortcomings such as long-term drift, lack of positional control, and operating in a constant gain mode. The modular design and open architecture provide a solid foundation for further development of the system. The work successfully bridges the theoretical control principles with practical implementation, offering significant educational value and a practical benchmark for exploring more advanced algorithms.

References

[1] L. G. B. Putra, F. Wahab, and T. A. Tamba, "Design and implementation of linear quadratic regulator control for two-wheeled self-balancing robot," *Bulletin of Electrical Engineering and Informatics*, vol. 14, no. 2, pp. 931–939, 2025.

[2] H. Zhang and N. Mohamad Nor, "Control strategies for two-wheeled self-balancing robotic systems: a comprehensive review," *Robotics*, vol. 14, no. 8, p. 101, 2025.

[3] N. Kumar, U. Arora, and N. Chaudhary, "Dynamic equilibrium in robotics: Techniques and applications for self balancing robots," in *2025 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS)*. IEEE, 2025, pp. 1–6.

[4] K. D. Papadimitriou, N. Murasovs, M. E. Giannaccini, and S. Aphale, "Genetic algorithm-based control of a two-wheeled self-balancing robot," *Journal of Intelligent & Robotic Systems*, vol. 111, no. 1, pp. 1–20, 2025.

[5] K. Ogata, *Modern control engineering*. Prentice hall, 2010.

[6] R. C. D. R. H. Bishop, *Modern control systems*, 2011.

[7] C. Savithri, R. Roopesh, N. Lavanya Devi, P. Shanthakumar, and P. Thirumurugan, "Self-balancing robot using arduino and pid controller," in *Computer Vision and Machine Intelligence Paradigms for SDGs: Select Proceedings of ICRTAC-CVMIP 2021*. Springer, 2023, pp. 201–208.

[8] A. S. Shekhawat and Y. Rohilla, "Design and control of two-wheeled self-balancing robot using arduino," in *2020 International Conference on Smart Electronics and Communication (ICOSEC)*. IEEE, 2020, pp. 1025–1030.

[9] L. B. Lau, N. S. Ahmad, and P. Goh, "Self-balancing robot: modeling and comparative analysis between pid and linear quadratic regulator," *International Journal of Reconfigurable and Embedded Systems*, vol. 12, no. 3, p. 351, 2023.

[10] H. Liao, Y. Liu, and L. Shi, "Adaptive robust control of wheel-legged self-balancing robot based on lqr," in *2025 44th Chinese Control Conference (CCC)*. IEEE, 2025, pp. 1–6.

[11] O. Saleem and J. Iqbal, "Fuzzy-immune-regulated adaptive degree-of-stability lqr for a self-balancing robotic mechanism: Design and hil realization," *IEEE Robotics and Automation Letters*, vol. 8, no. 8, pp. 4577–4584, 2023.

[12] S. Dash, S. S. Achary, S. Swain, and M. C. Tripathy, "Fopid-based self-balancing robot using advanced machine learning," in *2025 7th International Conference on Energy, Power and Environment (ICEPE)*. IEEE, 2025, pp. 1–6.

[13] R. Hernández, R. García-Hernández, and F. Jurado, "Stabilization of a two-wheeled self-balancing robot using reinforcement learning," in *2024 XXVI Robotics Mexican Congress (COMRob)*. IEEE, 2024, pp. 117–122.

[14] B. Firman, "Implementasi sensor imu mpu6050 berbasis serial i2c pada self-balancing robot," *Jurnal Teknologi Technoscientia*, pp. 18–24, 2016.

[15] B. Fan, L. Zhang, S. Cai, M. Du, T. Liu, Q. Li, and P. Shull, "Influence of sampling rate on wearable imu orientation estimation accuracy for human movement analysis," *Sensors*, vol. 25, no. 7, p. 1976, 2025.

[16] A. Ussayeva and Y. Tirkeshov, "Real-time controlled robot with gyroscope and accelerometer," *XXI* , no. 62-4 (1), 2025.

[17] P. Chotikunnan, W. Khotakham, A. Ma'arif, A. Nirapai, K. Javana, P. Pisa, P. Thajai, S. Keawkao, K. Roongprasert, R. Chotikunnan *et al.*, "Comparative analysis of sensor fusion for angle estimation using kalman and complementary filters," *International Journal of Robotics and Control Systems*, vol. 5, no. 1, pp. 1–21, 2025.